Section 8: Complete Cheat Sheet and Course Closing

This section is part of the 5-hour LLD course by Mukul Raina

- YouTube Course: https://www.youtube.com/watch?v=VSABVEBDhf0
- YouTube Channel: https://www.youtube.com/@mukulraina
- Instructor LinkedIn: https://www.linkedin.com/in/mukulraina/

SLIDE 97: Section Intro: Complete Cheat Sheet and Course Closing

SLIDE 98: OOP Principles Quick Reference

SLIDE 99: SOLID Principles Quick Reference

SLIDE 100: Creational Patterns Quick Reference

SLIDE 101: Structural Patterns Quick Reference

SLIDE 102: Behavioral Patterns Quick Reference

SLIDE 103: Complete Signal-to-Pattern Mapping

SLIDE 104: Pattern Combination Quick Reference

SLIDE 105: Anti-Pattern Recognition Quick Reference

SLIDE 106: Key Design Trade-Offs Summary

SLIDE 107: The LLD Interview Checklist

SLIDE 108: Interview Practice: Full LLD Walkthrough

SLIDE 109: What to Practice Next

SLIDE 110: Note It Down: The LLD Exam Day Reference

SLIDE 111: Bridge to Part 2: LLD Case Studies

SLIDE 97: Section Intro: Complete Cheat Sheet and Course Closing

Section 8: Complete Cheat Sheet and Course Closing

This section compiles every key concept, decision framework, and recognition signal from the entire course into a set of dense reference slides. No new concepts are introduced. Every slide here is designed to be screenshotted, bookmarked, or printed as a study aid for the days and hours before your LLD interview. Use these slides for final review, not for first-time learning. If anything on these slides is unfamiliar, go back to the section where it was taught.

What This Section Contains

1. OOP Principles quick reference
2. SOLID Principles quick reference
3. Creational, Structural, and Behavioral patterns quick reference
4. Complete signal-to-pattern mapping across all 14 patterns
5. Pattern combination reference
6. Anti-pattern recognition guide
7. Key design trade-offs summary
8. The LLD interview checklist
9. What to practice next
10. Bridge to Part 2 (LLD Case Studies)

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 98: OOP Principles Quick Reference

The Five OOP Decisions in Every LLD Interview

Decision Point	Question to Ask	Wrong Signal	Right Signal
Encapsulation	“Should external code change this field directly?”	Getter + setter on every field	Behavior methods that enforce rules

Decision Point	Question to Ask	Wrong Signal	Right Signal
Abstraction	“If I swap the implementation, does caller code change?”	Callers import concrete classes	Callers depend on interfaces
Inheritance	“Is this truly is-a? Does the child use most of the parent?”	Inherits for code reuse only	True is-a with shared behavior + state
Polymorphism	“Is there an if/else on object type?”	Switch statement selecting behavior	Interface with one implementation per type
Composition over Inheritance	“Are there 2+ independent dimensions of variation?”	One deep inheritance tree	Composed objects varying independently

Key Code Patterns

Concept	What It Looks Like in Java
Encapsulation	Private fields, no setters, behavior methods: `withdraw()`, `authenticate()`, `changePassword()`
Abstraction	`PaymentProcessor` interface, `StripePaymentProcessor` implementation
Polymorphism	`processor.charge(amount)` calls different code depending on the concrete type
Composition	`NotificationSender` holds `DeliveryMethod` + `PriorityHandler` + `MessageFormatter` as constructor parameters
Interface vs Abstract Class	Shared state (fields) = abstract class. Everything else = interface

Interview Relevance: These five decisions are evaluated in every LLD interview, whether the interviewer names them explicitly or not. Your class diagram either demonstrates them or violates them.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

SLIDE 99: SOLID Principles Quick Reference

One-Line Violation Test for Each Principle

Principle	Violation Test	If Violated
SRP	“How many reasons does this class have to change?”	More than one = split by change driver
OCP	“To add a new type, do I edit existing code?”	Yes = extract interface, one implementation per type
LSP	“Can I substitute a subtype without breaking behavior?”	No = fix the hierarchy or split the interface
ISP	“Does any implementor throw UnsupportedOperationException?”	Yes = split the interface by capability
DIP	“Does business logic import concrete infrastructure?”	Yes = depend on an interface, inject implementation

SOLID Violation Code Smells

Code Smell	Likely Violation	Refactoring
Class with 10+ methods across different domains	SRP	Extract focused service classes
Switch/if-else on type to decide behavior	OCP	Strategy pattern with interface
Subclass overrides method to throw exception	LSP + ISP	Split interface, fix hierarchy
Interface with 8+ methods	ISP	Split into role-based interfaces
Constructor creates dependencies with `new`	DIP	Constructor injection with interfaces
Business class imports SMTP/JDBC/HTTP client	DIP	Wrap behind your own interface

Interview Relevance: Spotting these code smells in your own design (or in code the interviewer presents) and naming the SOLID principle they violate is one of the strongest signals of design maturity.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

SLIDE 100: Creational Patterns Quick Reference

Pattern	Recognition Signal	Structure	Key Benefit
Factory Method	"Create different types based on configuration or input"	Interface + implementations + factory class/method	Adding a new type = one new class, factory update only
Abstract Factory	"Create a family of objects that must be consistent"	Factory interface + factory implementations, each producing a consistent set	Guarantees family consistency (all Material, all Fluent)
Builder	"5+ parameters, many optional, cross-field validation"	Inner Builder class with chained methods + `build()`	Self-documenting construction, immutable objects
Singleton	"Exactly one instance for correctness (not convenience)"	Private constructor, static access (prefer DI instead)	Single instance guarantee; trades testability for global access

Decision Shortcuts

Question	Answer
"Multiple types at runtime?"	Factory Method
"Objects must be created as a consistent set?"	Abstract Factory
"Too many constructor parameters?"	Builder
"Global access needed?"	Dependency injection (not Singleton)
"Only one type, few parameters?"	Simple constructor (no pattern needed)

Interview Relevance: Factory Method and Builder are the two creational patterns you will use most in LLD interviews. Abstract Factory is occasional. Singleton should be justified with a correctness argument, never a convenience argument.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

SLIDE 101: Structural Patterns Quick Reference

Pattern	Recognition Signal	Structure	Key Benefit
Adapter	“External API has incompatible interface”	Your interface + adapter class + third-party class	Isolates your code from external changes
Decorator	“Add behavior without modifying the core class”	Shared interface + base class + decorator wrappers	Stackable, n behaviors = n classes (not 2^n)
Facade	“Coordinate 3+ services behind a simple API”	Facade class orchestrating multiple services	Centralized sequencing and error recovery
Proxy	“Cache / lazy-load / check permissions”	Same interface as real object + proxy wrapper	Transparent access control

Structural Pattern Differentiation

Confusion Pair	Key Difference
Decorator vs Proxy	Decorator adds behavior. Proxy controls access.
Adapter vs Facade	Adapter translates one interface. Facade simplifies a subsystem of many interfaces.
Decorator vs Adapter	Decorator keeps the same interface. Adapter changes the interface.
Facade vs Adapter	Facade creates a new, simpler API. Adapter maps between two existing APIs.

Interview Relevance: Adapter and Decorator are the most frequently tested structural patterns. Proxy appears in caching and lazy-loading scenarios. Facade appears in multi-service coordination problems.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

SLIDE 102: Behavioral Patterns Quick Reference

Pattern	Recognition Signal	Structure	Key Benefit
Strategy	“Support multiple algorithms for X”	Context + strategy interface + implementations	Swap algorithms at runtime, OCP compliant
Observer	“When X happens, notify Y and Z”	Subject + listener interface + listener implementations	Decouple event source from reactions
Command	“Undo/redo, queue, or log operations”	Command interface with execute()/undo() + history/queue	Actions as first-class objects
State	“Object behaves differently by state”	Context + state interface + state implementations	Replace if/else on state, explicit transitions
Chain of Responsibility	“Process through pipeline of checks”	Handler interface + concrete handlers linked in chain	Sequential processing with short-circuiting
Template Method	“Same structure, different steps”	Abstract base with final skeleton method + abstract step methods	Shared algorithm, customizable steps

Behavioral Pattern Differentiation

Confusion Pair	Key Difference
Strategy vs State	Strategy: algorithm selected once, stays fixed. State: behavior changes as object transitions through lifecycle.
Observer vs Chain of Responsibility	Observer: all listeners notified independently. CoR: handlers process sequentially, can short-circuit.
Command vs Strategy	Command: action as object (store, undo, queue). Strategy: algorithm as object (swap, select).
Template Method vs Strategy	Template Method: fixed structure, variable steps (inheritance). Strategy: entire algorithm varies (composition).

Interview Relevance: Strategy, Observer, and State are the three behavioral patterns tested in 80%+ of LLD interviews. Command and Chain of Responsibility are Tier 2. Template Method is Tier 3.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

SLIDE 103: Complete Signal-to-Pattern Mapping

This is the master table. Every signal from every section in one place.

"What Varies?" Signals

Signal	Pattern
Multiple algorithms for the same operation	Strategy
Object behavior changes by lifecycle state	State
Different object types created at runtime	Factory Method
Family of objects that must be consistent	Abstract Factory
Same algorithm structure, different step implementations	Template Method

"What Reacts?" Signals

Signal	Pattern
Multiple independent reactions to one event	Observer
Action needs to be stored, queued, or undone	Command
Request processed through sequential checks	Chain of Responsibility

"What Wraps?" Signals

Signal	Pattern
External API has incompatible interface	Adapter
Add behavior without modifying the class	Decorator
Coordinate multiple services behind one API	Facade
Cache, lazy-load, or control access	Proxy

"How Is It Built?" Signals

Signal	Pattern
5+ parameters with many optional	Builder
Exactly one instance for correctness	Singleton (prefer DIP)

Interview Relevance: Internalizing this table is the single highest-leverage LLD interview preparation. When you hear a requirement, the pattern should surface automatically.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 104: Pattern Combination Quick Reference

Combination	When It Appears	How They Interact
Strategy + Factory	Multiple algorithms selected by configuration	Factory decides which strategy, strategy executes the algorithm
Observer + Command	Events trigger deferred/retryable work	Observer detects event synchronously, Command queued for async execution
State + Observer	State transitions trigger notifications	State class changes state, fires event for decoupled listeners
Decorator + Strategy	Cross-cutting concerns on any algorithm	Decorator wraps strategy interface, adds logging/timing/caching
Chain of Responsibility + Factory	Pipeline handlers vary by request type	Factory creates the handler chain, CoR processes sequentially
Template Method + Strategy	Fixed pipeline with swappable individual steps	Template Method owns the sequence, each step is a Strategy
Adapter + Factory	Multiple external integrations	Factory selects the right adapter based on provider configuration
Builder + Factory	Factory creates complex objects	Factory decides the type, Builder handles multi-parameter construction

The Two-Pattern Rule of Thumb

Most LLD interview problems are well-served by 2-3 patterns. If you find yourself reaching for a fourth or fifth pattern, pause and ask: “Is this solving a stated requirement or a hypothetical future one?” If the latter, remove it.

Interview Relevance: Knowing which patterns combine naturally prevents awkward designs where patterns fight each other. Each combination in this table has been validated through the worked examples in Sections 4-7.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 105: Anti-Pattern Recognition Quick Reference

Anti-Pattern	Detection Signal	SOLID Violation	Fix
God Class	8+ dependencies, 15+ methods across domains	SRP	Split by change driver into focused classes
Feature Envy	Method calls 3+ getters on another class	Encapsulation	Move method to the class it envies
Shotgun Surgery	One change requires editing 8+ files	SRP (inverted)	Consolidate scattered responsibility
Circular Dependency	Class A depends on B, B depends on A	DIP	Extract interface, use events, or add coordinator
Premature Abstraction	Interface with exactly one implementation	Over-engineering	Remove interface until second implementation exists
Refused Bequest	Subclass overrides method to throw or no-op	LSP	Break hierarchy, use composition
Switch on Type	if/else or switch selecting behavior by type string	OCP	Extract interface with implementations
Inappropriate Intimacy	Two classes access each other's internals	Encapsulation	Extract shared interface or merge if truly one concept

Three-Second Check

Before submitting your LLD design, scan for these three things:

1. **Any class with 5+ dependencies?** Likely a God Class. Split it.

2. **Any if/else on object type?** Likely an OCP violation. Extract interface.
3. **Any subclass throwing UnsupportedOperationException?** Likely LSP + ISP violation. Split the interface.

Interview Relevance: Interviewers sometimes present a flawed design and ask you to critique it. This table gives you the vocabulary and the specific fix for every common flaw.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 106: Key Design Trade-Offs Summary

Every design decision in an LLD interview is a trade-off. Never present a choice as "the right answer." Always state what you gain and what you give up.

The Seven Core Trade-Offs

Trade-Off	Option A	Option B	Decision Criteria
Inheritance vs Composition	Simpler for true is-a with shared state	Flexible, decoupled, runtime-swappable	Default to composition unless all three inheritance conditions are met
Interface vs Abstract Class	Multiple implementation, no shared state	Shared state, partial implementation	Shared fields = abstract class. Everything else = interface
Strategy vs State	Algorithm selected once, stays fixed	Behavior changes over object lifetime	Does the object transition? State. Fixed selection? Strategy
Decorator vs Proxy	Adding behavior (logging, compression)	Controlling access (caching, auth, lazy load)	What is the primary purpose: enhance or control?
Facade vs No Facade	Simplifies multi-service coordination	Callers may need fine-grained control	Facade for the 80% common case, direct access for the 20%
Pattern vs No Pattern	Extensibility for stated requirements	Simplicity for stable requirements	Use pattern only when requirement demands

Trade-Off	Option A	Option B	Decision Criteria
			extensibility
DI vs Singleton	Testable, visible dependencies, loose coupling	Simple global access, no constructor threading	Default to DI unless creating two instances causes a correctness bug

How to State Trade-Offs in an Interview

"I chose [Option] because [specific requirement]. The trade-off is [what I give up], but that is acceptable here because [why it does not matter for this use case]."

Example: "I chose composition over inheritance because there are two independent dimensions of variation (channel and priority). The trade-off is slightly more boilerplate (constructor injection), but that is acceptable because it gives us runtime flexibility and avoids the class explosion problem."

Interview Relevance: Explicitly stating trade-offs is the single most effective way to signal senior-level thinking in an LLD interview. Interviewers note whether candidates acknowledge costs or present every decision as obviously correct.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 107: The LLD Interview Checklist

Use this checklist as a mental walkthrough in the final 2 minutes of your LLD interview to catch common mistakes before your time runs out.

Requirements Phase

- Did I ask at least 3 clarifying questions?
- Did I state my assumptions for anything ambiguous?
- Did I scope the design to the stated requirements, not hypothetical future ones?

Object Design Phase

- Does each class have a single responsibility (one change driver)?

- Are my classes encapsulated (behavior methods, not getters/setters for every field)?
- Do any of my classes have 5+ dependencies? (God Class signal)

Interface and Relationship Phase

- Did I use interfaces where there are multiple implementations?
- Do my dependency arrows point toward abstractions, not concrete classes?
- Did I avoid interfaces with a single implementation and no testability requirement?
- Is there an if/else or switch on type anywhere? (Extract interface)

Pattern Phase

- Can I name the specific requirement each pattern solves?
- Am I using 2-3 patterns, not 6?
- Did I consider State vs Strategy correctly (lifecycle vs fixed selection)?

Trade-Off Phase

- Did I state at least one trade-off I made?
- Did I mention one extensibility point for future requirements?
- Can I defend every design decision if the interviewer asks “why?”

Interview Relevance: Running through this checklist silently in your head during the last 2 minutes of an interview catches the most common mistakes that downgrade LLD scores from “strong hire” to “hire” or below.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 108: Interview Practice: Full LLD Walkthrough

Scenario

You are in a 45-minute LLD interview. The interviewer says: “Design a library management system. Users can search for books, borrow books, return books,

and reserve books that are currently checked out. Librarians can add new books, remove books, and view overdue items."

Pause the video and walk through your five-phase approach for 3 minutes before continuing.

Phase 1: Clarify (3 min)

"Can a user borrow multiple books? Is there a maximum? Can books have multiple copies? Does a reservation expire? How are overdue books handled, just a report or automated fines?"

Phase 2: Core Objects (3 min)

`Book`, `BookCopy`, `User`, `Librarian`, `BorrowRecord`, `Reservation`, `LibraryService`.

Phase 3: Relationships and Patterns (7 min)

Book copies transition through states: Available, Borrowed, Reserved, Lost. This is the State pattern. When a book is returned, the system should notify anyone who reserved it, update inventory counts, and check for overdue fines. This is Observer. Search functionality across title, author, and ISBN does not need a pattern since it is a query method, not a dimension of variation.

Phase 4: Key Classes (15 min)

Detail `BookCopyState` interface with `AvailableState`, `BorrowedState`, `ReservedState`. Show `BorrowRecord` with due date and overdue detection. Show `LibraryEventListener` with `ReservationNotifier` and `FineCalculator` implementations.

Phase 5: Extensibility (5 min)

"If we needed multiple search strategies (keyword, full-text, fuzzy match), I would extract a `SearchStrategy` interface. If we needed to support different borrowing policies by user tier (student, faculty, public), I would use Strategy for `BorrowPolicy`."

Pattern Count: 2 (State + Observer). This is sufficient for the stated requirements. Adding more would be over-engineering.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

SLIDE 109: What to Practice Next

Immediate Next Steps

1. **Practice the five-phase structure** on 3 different LLD problems with a 45-minute timer. Record yourself if possible. Most improvement comes from fixing timing issues.
2. **Build the signal-to-pattern reflex.** Take any product you use daily (Spotify, Uber, Gmail) and identify 3 requirements that map to specific patterns. Practice until the pattern surfaces within 5 seconds of hearing the signal.
3. **Practice anti-pattern recognition.** Open any codebase you have access to and find one God Class, one Feature Envy method, and one if/else chain that should be a Strategy. Real code builds recognition faster than textbook examples.

LLD Problems to Practice (Ordered by Pattern Density)

Problem	Primary Patterns	Difficulty
Parking Lot	State, Strategy (if multi-strategy), Factory	Beginner
Library Management	State, Observer	Beginner
Vending Machine	State, Factory	Beginner
Elevator System	State, Strategy, Observer	Intermediate
ATM Machine	State, Chain of Responsibility, Command	Intermediate
Online Book Store	Strategy, Factory, Observer, Facade	Intermediate
Chess Game	State, Strategy, Command (undo), Observer	Advanced
Hotel Booking	State, Strategy, Observer, Builder	Advanced
Ride-Sharing (like Uber)	Strategy, State, Observer, Factory, Decorator	Advanced
Food Delivery (like DoorDash)	Strategy, State, Observer, Chain of Responsibility, Command	Advanced

Interview Relevance: The problems in this table are the most commonly asked LLD interview questions across Amazon, Google, Microsoft, Meta, and Uber. Practicing 5-6 of these with the five-phase structure covers the majority of what you will encounter.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 110: Note It Down: The LLD Exam Day Reference

This is the final “note it down” slide. Review this 30 minutes before your LLD interview.

The Three Questions (Pattern Selection)

1. What varies? (Strategy, State, Factory, Template Method)
2. What reacts? (Observer, Command, Chain of Responsibility)
3. What wraps? (Decorator, Proxy, Adapter, Facade)

The Five Phases (Interview Structure)

1. Clarify requirements (3-5 min)
2. Identify core objects (3-5 min)
3. Define interfaces and relationships (5-10 min)
4. Detailed class design (10-15 min)
5. Extensibility and trade-offs (5 min)

The Three Danger Checks (Before You Finish)

1. Any class with 5+ dependencies? Split it.
2. Any if/else on type? Extract interface.
3. Any subclass throwing exceptions? Split the interface.

The One-Line Trade-Off Template

“I chose [X] over [Y] because [specific requirement]. The trade-off is [cost], which is acceptable because [justification].”

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 111: Bridge to Part 2: LLD Case Studies

This course covered the foundations: OOP principles, SOLID principles, 14 design patterns across three categories, pattern selection frameworks, and anti-pattern recognition. These are the building blocks that every LLD interview answer is constructed from.

What Part 2 Will Cover

Part 2 applies everything from this course to complete, end-to-end LLD case studies. Each case study will:

1. Start with an interview-style problem statement
2. Walk through all five phases of the LLD interview structure
3. Produce a complete class diagram with interfaces, implementations, and relationships
4. Write full Java implementations of the key classes
5. Discuss trade-offs and extensibility points
6. Show what a “hire,” “strong hire,” and “weak” answer look like for the same problem

Planned Case Studies

Case Study	Key Patterns	Key Trade-Offs
Parking Lot System	State, Strategy, Factory	Single vs multi-floor, pricing strategy
Elevator Design	State, Strategy, Observer, Command	Scheduling algorithms, concurrent requests
Online Bookstore	Strategy, Factory, Observer, Facade, Decorator	Payment flow, discount composition
Chess / Board Game	State, Command, Strategy, Observer	Move validation, undo/redo, AI player
Rate Limiter	Strategy, Chain of Responsibility, Proxy	Token bucket vs sliding window, distributed vs local
Notification Service	Strategy, Factory, Observer, Decorator, Adapter	Channel selection, retry, template rendering

Each case study is a complete worked example that demonstrates how to combine the patterns from this course into a cohesive design under interview conditions.

How to Use This Course and Part 2 Together

This course (Part 1) gives you the vocabulary and the decision frameworks. Part 2 gives you the practice. Watch this course first, practice the signal-to-pattern recognition until it is automatic, then work through Part 2 to build fluency in applying the patterns under time pressure.

Interview Relevance: Part 1 teaches you to recognize patterns. Part 2 teaches you to apply them fluently under interview conditions. Both are necessary for consistent LLD interview performance.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn